

ENTREGABLE 2.1 – MANIPULACIÓN DE IMÁGENES CON MATLAB

Apartado 1.

Genere y represente una imagen binaria de tamaño 256×256 correspondiente la máscara que se muestra en la figura 1. Los radios de la corona circular deben ser 32 y 128 píxeles para las circunferencias interior y exterior respectivamente.

Para lograr conseguir esto hemos seguido el siguiente procedimiento:

Definimos el tamaño de la matriz cuadrada, que será de 256×256 :

size = 256;

Siendo *a* el radio interior y *b* el radio exterior, estos valores determinan los límites entre los cuales la máscara circular será blanca:

radio_a = 32 y radio_b = 128;

Se crea la matriz *mask* de 256×256 , inicialmente cada pixel tendrá valor 0 (negro), y más adelante se actualizarán los valores a 1 (blanco):

mask = zeros(size);

Calculamos el centro de la imagen dividiendo el tamaño de esta entre 2, desde este punto mediremos la distancia a cada píxel para determinar si está dentro o fuera de la corona circular:

center = size / 2;

Utilizamos un sobre bucle *for* para recorrer toda la matriz:

for x = 1:size

for y = 1:size

Para cada píxel, se calcula la distancia al centro de la imagen:

dist = sqrt((x - center)^2 + (y - center)^2);

Una vez tenemos la distancia, se evalúa si está contenida entre los radios, en caso afirmativo el pixel se pone a uno:

if dist >= radio_a && dist <= radio_b

mask(x, y) = 1;

end

Finalmente mostramos la matriz aplicándole un mapa de color “gray”, que representa los 0 como negros y los 1 como blancos:

```
imshow(mask);
```

```
colormap(gray);
```

```
title('Máscara binaria con radios 32 y 128 píxeles');
```

Apartado 2.

Genere ahora una carta de colores 256×256 como la mostrada en la figura 2. Para ello, debe trabajar en el espacio HSV. El canal de tinte variará en diagonal para todo su rango siguiendo la estructura de una matriz de Toeplitz (comando `toeplitz` de MATLAB). Luminancia y saturación se mantendrán constantes en su valor máximo.

Volvemos a definir el tamaño de la matriz cuadrada de 256x256 igual que antes

Ahora con `linspace` creamos un vector con size 256 que varían linealmente desde 1 hasta 0, correspondiendo a un tinte de 360 grados en HSV (rojo) hasta uno de 0 grados (también rojo), los valores intermedios cubren todo el rango de tonalidades intermedias.

Con `toeplitz`, creamos una matriz de toeplitz en hue, con constantes en las diagonales, en este caso el resultado es una matriz de 256x256 donde el tinte varía en diagonal.

```
hue = toeplitz(linspace(1, 0, size));
```

Ahora definimos los canales de saturación y de valor, ambas con una matriz de unos, lo que significa que la saturación y el valor son máximos en todos los píxeles, lo que garantiza colores completamente saturados y máxima luminosidad en todos los canales:

```
saturation = ones(size);
```

```
value = ones(size);
```

Ahora concatenamos los tres canales usando `cat`, formando una imagen HSV.

```
hsv_img = cat(3, hue, saturation, value);
```

Para representarla debemos pasarlo a RGB, y para eso usamos la función:

```
rgb_img = hsv2rgb(hsv_img);
```

Por último visualizamos la imagen:

`imshow(rgb_img);`

Apartado 3.

Repita el ejercicio anterior pero haga variar el canal de saturación con el mismo patrón que el de tinte, aunque en este caso el rango variará en el intervalo [0.5, 1] (ver figura 3). Represente la imagen y comente las diferencias obtenidas.

Este apartado es igual que el anterior, con la única modificación:

`saturation = 0.5 + 0.5 * toeplitz(linspace(0, 1, size));`

Con `linspace` generaremos un vector de valores espaciados uniformemente desde 0 hasta 1, es decir, valores de saturación máxima y saturación mínima (gris). Luego convertimos este vector, y lo convertimos en una matrix de Toeplitz, explicada anteriormente.

Por último se aplica la operación $0.5 + 0.5 * (...)$ para ajustar los valores de saturación a un rango entre 0.5 y 1, es decir, cuando la matriz sea 0 en la matriz de Toeplitz, el valor mínimo de la saturación será 0.5.

Es decir, al realizar esa operación, estamos desplazando y escalando el rango de saturación, de manera que garantizamos que todos los píxeles tengan al menos un 50% de saturación para evitar valores de saturación muy bajos.

Apartado 4.

Utilice la orden `rgb2ind` para cuantificar la última imagen obtenida con una profundidad de 3 bits. Utilice las opciones del comando para aplicar una cuantificación de mínima varianza y aplíquela con y sin dithering. Explique las diferencias obtenidas comparando con la imagen original.

Lo primero que hacemos es convertir una imagen RGB en una imagen indexada con 2^3 colores con la función `rgb2ind`. El parámetro 'nodither' indica que no se aplicará dithering, lo que significa que no habrá dispersión de los errores de cuantización para suavizar los bordes o las transiciones entre colores. `map_no_dither` es el mapa de colores que contiene los 8 colores de la imagen indexada.

`[indexed_image_no_dither, map_no_dither] = rgb2ind(rgb_img, 8, 'nodither');`

Ahora configuramos una cuadrícula de gráficos de 1 fila y 2 columnas:
subplot(1, 2, 1);

Ahora mostramos la primera imagen indexada sin dithering, aplicando el mapa de colores antes mencionado:

imshow(indexed_image_no_dither, map_no_dither);

Ahora haremos exactamente lo mismo que antes, pero con el parámetro 'dither' en vez de 'nodither' en la función `rgb2ind`, lo cual nos devolverá la misma imagen que antes, pero con dithering, posteriormente la representaremos de la misma manera que la anterior.

Apartado 5.

Aplique la máscara obtenida en el apartado 1 a las imágenes cuantificadas, de forma que sólo se visualice la región correspondiente a la corona circular. La figura 4 muestra un ejemplo para la imagen sin cuantificar.

Creamos una copia de la imagen cuantificada sin dithering en una nueva variable:

masked_image_no_dither = indexed_image_no_dither;

Luego aplicamos la máscara creada en el Apartado 1, haciendo que todos los píxeles que sean 0 en la máscara sean cero también en la imagen, es decir, negro.

masked_image_no_dither(~mask) = 0;

Creamos otra copia de la imagen cuantificada con dithering, y le aplicamos la máscara de la misma manera que a la anterior.

Por último, creamos el subplot y mostramos las imágenes finales enmascaradas.

Apartado 6.

Repita el ejercicio 4 para la imagen enmascarada.

Primero aplicamos la máscara a la imagen original, esta vez, con la función `bsxfun`. Esta función aplica una operación de manera vectorizada entre matrices del mismo tamaño, en este caso, cada píxel de `rgb_img` con `mask`. Con `cast`, convertimos la máscara a un tipo compatible con `rgb_img` para poder realizar la operación:

```
masked_rgb_img = bsxfun(@times, rgb_img, cast(mask, 'like', rgb_img));
```

Luego realizamos la cuantificación sin dithering de manera análoga a apartados anteriores, y la mostramos:

```
[indexed_image_no_dither_masked, map_no_dither_masked] =  
rgb2ind(masked_rgb_img, 8, 'nodither');
```

Y por último la cuantificación con dithering, y la mostramos:

```
[indexed_image_dither_masked, map_dither_masked] =  
rgb2ind(masked_rgb_img, 8, 'dither');
```